Programación Backend
Evaluación Sumativa 3
Instrucciones para la evaluación.

Instrucciones

Esta evaluación es una **continuación de ES01 y ES02**. Debes evolucionar el mismo proyecto Django que has venido trabajando (o, si es necesario, migrar a otro de los casos propuestos manteniendo la misma complejidad). La evaluación se realiza **en parejas** para priorizar la entrega de resultados y las propuestas de los estudiantes para avanzar a la evaluación número 4.

Al finalizar esta evaluación, el estudiante será capaz de:

1. **Configurar Django REST Framework** según la documentación oficial, exponiendo los modelos de su proyecto a través de endpoints REST.
2. **Diseñar e implementar Serializers y ViewSets** coherentes con los modelos y reglas del dominio trabajados en las unidades 1 y 2.
3. **Configurar autenticación JWT con SimpleJWT**, permitiendo el acceso seguro a la API por medio de encabezados HTTP (Bearer Token).
4. **Configurar CORS** utilizando django-cors-headers para habilitar el consumo de la API desde una aplicación cliente externa.
5. **Construir un cliente (Web o Aplicación Nativa)** que consuma los endpoints de la API REST, integrándose funcionalmente con la solución existente (mejorando vistas previas o agregando una solución complementaria).

Estos objetivos se vinculan con los criterios del descriptor: 3.1.1–3.1.4 (configuración de DRF, autenticación, generación de JSON y construcción de una API Restful funcional). Tu organización ha evolucionado su aplicación web interna (desarrollada en ES01 y ES02) y ahora requiere: **Exponer servicios mediante API REST**, **Asegurar el acceso con JWT, Permitir el acceso desde otros dominios o dispositivos** para finalmente **Proporcionar un cliente funcional (Web/Python/Arduino)**.

Requerimientos funcionales y técnicos (Backend – API REST con DRF y JWT)

1. **Configuración de Django REST-Framework**
    - Instalar y configurar djangorestframework.
    - Definir REST_FRAMEWORK en settings.py, incluyendo **clases de autenticación y permisos por defecto**.
    - Usar **routers de DRF** para registrar los ViewSets principales.
2. **Serializers y ViewSets**
    - Crear **Serializers** para los modelos principales del proyecto (al menos los usados en el flujo central del caso).
    - Crear **ViewSets** que permitan al menos las operaciones CRUD básicas sobre dichos modelos, respetando la lógica de negocio definida previamente.
    - Garantizar que las respuestas sean en formato **JSON** y coherentes con una API RESTful (nombres de campos, códigos de estado HTTP, etc.).
3. **Autenticación con SimpleJWT**
    - Instalar y configurar djangorestframework-simplejwt.
    - Definir las vistas de obtención y refresco de token (/api/token/, /api/token/refresh/).
    - Configurar DEFAULT_AUTHENTICATION_CLASSES para usar JWTAuthentication.
4. **Configuración de CORS**
    - Instalar y configurar django-cors-headers.
    - Permitir explícitamente el origen del cliente (dominio o localhost y puerto que usarás).
    - Evitar configuraciones excesivamente abiertas (CORS_ALLOW_ALL_ORIGINS=True sólo como último recurso justificado).
5. **Despliegue**
    - Mantener o actualizar el **despliegue en la nube** (Vercel + Supabase) ya implementado en ES02, incorporando la API REST y la configuración de JWT/CORS.
    - Hay que asegurar que la **URL pública** de la API REST sea accesible y se muestre en el README del proyecto.

Requerimientos funcionales y técnicos (Cliente consumidor: Web, Python o Arduino)

Debes desarrollar un **proyecto cliente** que consuma tu API REST. Puedes elegir una de las siguientes líneas:

1. **Aplicación Web (recomendado)**. Puede ser una nueva app frontend (por ejemplo, HTML/JS, framework frontend o incluso vistas Django que consuman la API mediante fetch o el paquete npm axios).
    - Debe de cumplir con:
        - Autenticarse contra la API usando JWT (obtener token con usuario/contraseña).
        - Enviar el **Bearer Token** en el header Authorization en las peticiones protegidas.
        - Crear una interfaz que incluya una UI/UX responsive con librerías para expandir los datos mostrados, ya sea modelos 3D con Three.JS, animaciones, skeletons, loaders, etc.
2. **Cliente Nativo.** Dispositivo móvil o nativo que consuma uno o más endpoints de la API, por lo que el flujo mínimo debe incluir:
        - Uso de un token JWT (o credenciales para obtenerlo desde otro servicio intermedio) y una llamada a la API.
        - Permita guardar y hacer persistencia de datos offline de la data disponible previamente

---

*El cliente debe **aportar valor** a la solución: puede ser una versión mejorada de las vistas tradicionales ya desarrolladas o una solución complementaria (monitoreo en tiempo real, interfaz para otro tipo de usuario, etc.).*

Entregables: Repositorio Backend (Django)

- Código fuente con:
  - Configuración DRF y SimpleJWT.
  - Serializers, ViewSets y routers.
  - Configuración de CORS.

Entregables: Repositorio del Cliente

- Código fuente del cliente (Web o Nativo).
- Instrucciones para ejecutarlo.
- Explicación de qué problema resuelve y cómo usa la API.

Colección de pruebas (opcional pero recomendable)

- Scripts curl que demuestren el uso de los endpoints (con y sin autenticación).

Indicadores	Siempre	A veces	Nunca
Configura Django REST Framework según la documentación oficial (3.1.1).	6 puntos	4 puntos	1 puntos
Implementa Serializers y ViewSets coherentes para los modelos clave del proyecto (3.1.3).	6 puntos	4 puntos	1 puntos
Define endpoints y rutas RESTful claros, consistentes y funcionales (3.1.4).	6 puntos	4 puntos	1 puntos
Configura SimpleJWT correctamente: obtención/refresh de tokens y protección de endpoints (3.1.2).	6 puntos	4 puntos	1 puntos
Cumple con agregar funcionalidades mínimas de las unidades previas.	6 puntos	4 puntos	1 puntos
Cumple con medidas mínimas de seguridad establecidas en clases.	6 puntos	4 puntos	1 puntos
Aplica permisos y políticas de acceso adecuados (IsAuthenticated, AllowAny, etc.) en la API.	6 puntos	4 puntos	1 puntos
Configura CORS con django-cors-headers de forma segura y funcional para el cliente.	6 puntos	4 puntos	1 puntos
Desarrolla un cliente (Web o Nativo) que consume la API usando JWT (Bearer Token).	6 puntos	4 puntos	1 puntos
Mantiene buena calidad de código y estructura de proyecto (organización, uso de variables de entorno, documentación básica).	6 puntos	4 puntos	1 puntos
Presenta un video explicativo claro, que demuestra el flujo end-to-end y las decisiones de diseño/seguridad.	6 puntos	4 puntos	1 puntos
Demuestra funcionamiento: URL pública operativa y flujo CRUD completo accesible.	6 puntos	4 puntos	1 puntos